

CS-200 — Processeur multicycle : suivi des slides 2a

Oct 9, 2026 · @Gab

Une mémoire synchrone et unifiée impose plusieurs cycles par instruction ; le datapath se construit ensuite instruction par instruction, et la FSM qui le pilote n'est que la table « état → valeurs des signaux de contrôle ». Cette fiche suit le cours CS-200, partie 2a (Paolo Ienne), slide par slide. Les passages marqués *interprétation* ne figurent pas explicitement dans les slides et sont à vérifier dans l'énoncé du labo.

1. Le cadre : ISA, processeur, mémoire (slides 2–5)

La mémoire unifiée n'a qu'un seul port d'adresse : c'est la première raison du multicycle.

Slide 2 – Le contrat HW/SW. L'ISA (ici RISC-V) est l'interface. Au-dessus, le logiciel ne voit que les instructions et leur sémantique. En dessous, le matériel réalise cette sémantique comme il veut. Le processeur multicycle est une *microarchitecture* parmi d'autres (single-cycle, pipeline) ; le programme assembleur ne voit aucune différence, hors timing.

Slide 3 – Le processeur. Les blocs : banc de registres (RF), ALU, mémoire de données, mémoire d'instructions, PC, logique de contrôle. Le schéma montre $PC + 1$ par simplification. RISC-V est adressé à l'octet avec des instructions de 4 octets, donc en réalité $PC + 4$.

Slide 4 – Mémoire unifiée. Instructions et données sont dans la même mémoire (architecture von Neumann). Les deux ports `Address` deviennent un seul port. Il faut donc un multiplexeur qui choisit entre le PC (lecture d'instruction) et le résultat de l'ALU ($1w / sw$). On ne peut pas lire l'instruction et accéder aux données dans le même cycle. Ce multiplexeur réapparaît plus tard sous le nom `sel_addr`.

Slide 5 – Une grande FSM. Le pointillé entoure le contrôle et le PC. Le processeur entier est une machine à états finis, dont l'état comprend le PC (et en toute rigueur les registres et la mémoire). La partie « contrôle » est la FSM au sens étroit.

2. Pourquoi plusieurs cycles par instruction (slides 6–11)

Couper le chemin critique ne fait rien gagner en soi ; la raison décisive est que les mémoires synchrones rendent toute lecture longue de deux cycles.

Slide 6 – Single-cycle. La FSM n'a qu'un état : « exécuter l'instruction, $PC \leftarrow PC + 4$ », en boucle jusqu'à `ebreak`.

Slide 7 – Chemin critique. La période est contrainte par le plus long chemin combinatoire entre deux registres :

$$T_{\text{clk}} \geq t_{\text{clk} \rightarrow q} + t_{\text{comb}, \max} + t_{\text{setup}}$$

En single-cycle, ce chemin traverse tout : PC → mémoire d'instructions → décodage → lecture RF → ALU → mémoire de données → mux → écriture RF. Le registre de départ et celui d'arrivée peuvent être le même (le PC).

Slides 8-9 – Couper le chemin en deux ? Un registre au milieu divise le chemin critique par deux et double la fréquence, mais chaque instruction prend 2 cycles. Avec N instructions :

$$T_{\text{exec}} = N \cdot \text{CPI} \cdot T_{\text{clk}}$$

$$N \cdot 2 \cdot \left(\frac{T}{2} + t_{\text{overhead}} \right) = N \cdot T + 2N \cdot t_{\text{overhead}} \quad ;> \quad N \cdot T$$

Réponse à « Did we gain anything? » : non. On perd même un peu, à cause du $t(\text{clk} \rightarrow q) + t(\text{setup})$ du registre ajouté et d'une coupure jamais parfaitement au milieu.

Slide 10 – Tous les chemins ne se valent pas. En single-cycle, la période est imposée par l'instruction la plus lente ($1w$), même pour un `andi` bien plus rapide. Le multicycle donne à chaque classe d'instructions i son propre nombre de cycles, de fréquence d'apparition f_i :

$$\text{CPI}_{\text{moyen}} = \sum_i f_i \cdot \text{CPI}_i \quad \text{gain} \quad \text{iff} \quad \text{CPI}_{\text{moyen}} \cdot T_c < T_{\text{single}}$$

Remarque : en pratique, ce gain en performance pure est souvent faible, voire nul, car les étages ne sont jamais parfaitement équilibrés. Le vrai gain viendra du pipeline. Ici, les raisons décisives sont la mémoire unifiée (slide 4) et la slide 11.

Slide 11 – Les mémoires sont séquentielles. C'est la slide clé. La mémoire est synchrone :

1. Au cycle 0, on présente l'adresse avec `cs` et `read`.
2. La mémoire échantillonne l'adresse au front d'horloge.
3. La donnée (`rddata`) n'est valide qu'au cycle 1.

Une lecture prend donc intrinsèquement deux cycles : lancer, puis récupérer. Avec ce type de mémoire (celui des FPGA), un processeur single-cycle est impossible. D'où les paires d'états « Start a Load / Complete a Load ».

3. La FSM multicycle (slides 12–17)

Six états suffisent pour les instructions vues : Fetch1, Fetch2, Decode, Execute, Load1, Load2.

Fetch1 → Fetch2 (slide 12). C'est le motif de la slide 11 appliqué à l'instruction. Fetch1 présente le PC comme adresse. Fetch2 voit l'instruction arriver sur `rdata` et l'enregistre dans le registre d'instruction (IR).

Decode (slide 13). L'instruction est stable dans IR ; le contrôleur lit l'opcode et choisit l'état suivant. Décoder dès Fetch2 allongerait le chemin critique : accès mémoire, puis décodage, puis logique d'état suivant, dans le même cycle.

Execute / ALU (slide 14). Une instruction simple comme `andi` se termine en un cycle : lecture RF → ALU → écriture RF.

Load1 → Load2 (slide 15). Un `lw` réutilise le motif de la slide 11. Load1 calcule l'adresse avec l'ALU et la présente à la mémoire. Load2 récupère la donnée et l'écrit dans le RF.

Slide 16 – Deux objectifs. Peu d'états, car chaque état de plus coûte un cycle. Des chemins équilibrés, car la période est imposée par le chemin le plus long parmi tous les états : un seul état trop chargé ralentit toutes les instructions.

Slide 17 – Mealy ou Moore ? En Moore, sorties = $g(\text{état})$. En Mealy, sorties = $g(\text{état}, \text{entrées})$. L'entrée principale de la FSM est l'instruction dans IR.

- **Moore** : une sortie qui dépend de l'instruction exige un état distinct. Cela peut coûter des cycles, parfois de façon inacceptable.
- **Mealy** : la sortie réagit dans le même cycle, mais crée un chemin combinatoire entrée → sortie qui peut allonger le chemin critique.

Ce compromis réapparaît concrètement à la slide 32 (section 5).

4. Construire le datapath instruction par instruction (slides 18–30)

On part d'un contrôleur vide et d'un PC (slide 18), puis on ajoute uniquement ce qu'exige chaque type d'instruction ; chaque ajout crée un signal de contrôle.

Slide 19 – IR et enables. IR garde l'instruction pendant toute son exécution, alors que `rdata` change (par exemple lors d'un `lw`). IR et PC ne se mettent à jour qu'à des moments précis, via des enables (`ir_en`, `pc_en`) produits par la FSM. En Verilog : `if (en) q <= d; .`

Slide 20. IR est relié au contrôleur, car l'état suivant après Decode dépend de l'opcode.

Slide 21 – Type I : `addi t0, t1, 1234`.

- `instr[19:15]` (`rs1`) va sur `aa` ; `instr[11:7]` (`rd`) va sur `aw`.
- La sortie `a` du RF va en entrée A de l'ALU, l'immédiat en entrée B ; le résultat va dans `wrdata`.
- Nouveaux signaux : `rf_we`, `op_alu` (6 bits) et `imm` (étendu en signe), produit par le contrôleur.

Slide 22 – Où s'arrête le contrôleur ? Il n'y a pas de réponse unique. Les choses triviales, comme extraire `instr[19:15]` pour le RF, sont plus lisibles hors de la FSM, sous forme de fils.

Slide 23 – Type R : `add t0, t1, t2`. `instr[24:20]` (`rs2`) va sur `ab`. Un mux `sel_b` choisit l'entrée B de l'ALU : immédiat (0) ou sortie `b` du RF (1).

Slide 24 – Type U : `lui t0, 1234`. L'immédiat est écrit directement dans le RF, sans ALU ; ici `t0 = 1234 << 12`. Un mux `sel_imm` choisit entre l'ALU (0) et l'immédiat (1).

Slide 25 – Adresse des load/store : `lw t0, 16(t1)`. L'adresse `rs1 + imm` est calculée par l'ALU (`sel_b = 0`, addition). Le mux `sel_addr` de la slide 4 apparaît : 0 = PC (fetch), 1 = résultat de l'ALU (load/store).

Slide 26 – Les loads écrivent `rdata`. Le mux `sel_mem` sélectionne `rdata` comme donnée à écrire dans le RF, en Load2.

Slide 27 – Stores : `sw t0, 16(t1)`. En RISC-V, c'est `sw rs2, imm(rs1)` : `t0` est `rs2`.

- La sortie `b` du RF part directement vers `wdata`, en contournant `sel_b`, car l'entrée B de l'ALU sert à l'immédiat d'adresse.
- La FSM met `we = 1` et `rf_we = 0`.
- L'immédiat du format S est découpé (`instr[31:25]` et `instr[11:7]`) : la génération d'immédiat dépend du format, d'où sa place dans le contrôleur.

Slide 28 – Branches : `beq t0, t1, 1234`. L'ALU fait la comparaison (`sel_b = 1`) et le bit 0 de `alu_res` porte le résultat. L'enable effectif du PC est :

```
en_{PC} = pc\_en \;\lor\; (branch\_op \land alu\_res_0)
```

Le PC devient un module avec `sel_pc_base`, `add_imm` et `imm` pour calculer `PC + imm`.

Interprétation : le PC est incrémenté pendant le fetch (la slide 29 lit `PC + 4` directement sur sa sortie). Or l'offset de branche RISC-V est relatif à l'adresse de l'instruction de branche. `sel_pc_base` choisirait donc la base : PC courant ou adresse de l'instruction en cours, que le module PC mémorise. À vérifier dans l'énoncé. Si la branche n'est pas prise, il n'y a rien à faire : le PC vaut déjà `PC + 4`.

Slide 29 – `jal ra, imm`. Deux écritures au même front : $rd \leftarrow PC + 4$ (valeur actuelle du PC) et $PC \leftarrow \text{adresse de l'instruction} + \text{imm}$. Le RF capture l'ancienne valeur du PC, car les registres échantillonnent juste avant le front (voir slide 39).

Interprétation du schéma : deux mux en cascade pour `wrdata`. Le premier (`sel_mem`) choisit entre `addr` du PC (0) et `rdata` (1). Le second choisit entre le chemin ALU/immédiat (0) et la sortie du premier (1), commandé par `sel_pc OR sel_mem`. On obtient trois sources : ALU/imm, mémoire, PC + 4.

Slide 30 – `jalr x0, 0(ra)` (c'est `ret`). La cible $rs1 + \text{imm}$ est calculée par l'ALU et envoyée au PC via son entrée `alu` (`sel_alu = 1`). $rd = x0$, donc PC + 4 est jeté. La spécification impose de mettre le bit 0 de la cible à zéro (`& ~1`), absent de la slide.

5. Retour à la FSM : signaux par état (slides 31–32)

Chaque état est défini par les valeurs des signaux de contrôle qu'il produit ; $PC \leftarrow PC + 4$ arrive exactement une fois par instruction.

Slide 31. Deux exemples donnés : `sel_addr = 0` en Fetch1 (adresse = PC) mais `sel_addr = 1` en Load1 (adresse = ALU). Les lignes marquées * ci-dessous sont une reconstitution logique, absente des slides.

État	Rôle	Signaux actifs
Fetch1	lancer la lecture de l'instruction	<code>sel_addr = 0</code>
Fetch2	capturer l'instruction	<code>ir_en = 1</code> , <code>pc_en = 1</code> * ($PC \leftarrow PC + 4$)
Decode	lire l'opcode, choisir l'état suivant	aucune écriture
Execute (ALU)	types I, R, U	<code>rf_we = 1</code> , <code>sel_b = 0</code> (I) ou 1 (R), <code>op_alu</code> d'après <code>funct3/funct7</code> , <code>sel_imm = 1</code> si lui
Execute (Branch)	comparer, sauter éventuellement	<code>sel_b = 1</code> , <code>branch_op = 1</code> , <code>pc_add_imm = 1</code>
Load1	présenter l'adresse $rs1 + \text{imm}$	<code>sel_addr = 1</code> , <code>sel_b = 0</code> , <code>op_alu = \text{addition}</code>
Load2	écrire la donnée lue	<code>rf_we = 1</code> , <code>sel_mem = 1</code>
Store*	écrire en mémoire	<code>sel_addr = 1</code> , <code>we = 1</code> , <code>sel_b = 0</code>

Slide 32 – Deux variantes équivalentes.

- **Un seul état Execute**, avec des sorties conditionnelles : `update_PC = 1 si Branch`, `rf_we = 1 si ALU`. C'est du Mealy : la sortie dépend de l'instruction.
- **Deux états** Execute-ALU et Execute-Branch, choisis dès Decode. Les sorties de chacun sont constantes (Moore) et on ne reteste pas en Execute ce que Decode a déjà testé.

Les deux sont correctes ; la seconde est souvent plus propre et plus facile à déboguer.

6. Méthodologie Verilog (slides 33–39)

On n'écrit le Verilog qu'une fois tout décomposé en registres simples, combinatoire et FSM formelle.

Slide 33 – Le schéma d'abord. Verilog *décrit* du matériel. Sans savoir exactement quel matériel on veut, le code sera faux d'une manière très difficile à déboguer.

Slides 34–35 – Classer chaque bloc avant de l'écrire.

Type de bloc	Exemples	Prêt à écrire ?
Séquentiel simple	IR, RF, registres	oui
Combinatoire simple	mux, portes	oui
FSM formelle (états et transitions dessinés)	contrôleur	oui
Séquentiel complexe	PC (additionneur, mux, registre d'adresse)	non : d'abord un diagramme interne
Combinatoire complexe	ALU	probablement non : un diagramme rend l'écriture plus sûre

Slide 36 – Décomposition de l'ALU. Quatre sous-unités : add/sub, comparateur, unité logique, shifter (qui n'utilise que `B[4:0]`). Un mux final commandé par `op[5:4]` choisit la sortie.

Dans l'additionneur, `B` passe par un XOR avec `sub`, et `sub` est aussi la retenue d'entrée. Si `sub = 1` :

$$A + \overline{B} + 1 = A + (2^{32} - 1 - B) + 1 = A - B + 2^{32}$$

C'est bien $A - B$ modulo 2^{32} en complément à deux. Le comparateur en déduit :

- **Égalité** : $A = B$ ssi `zero` = 1.
- **Non signé** : la retenue sortante vaut 1 ssi $A - B + 2^{32} \geq 2^{32}$, soit ssi $A \geq B$. Donc $A <_u B$ ssi `carry` = 0.
- **Signé, signes différents** ($A_{31} \neq B_{31}$) : $A < B$ ssi $A_{31} = 1$, car A est le négatif.
- **Signé, mêmes signes** : $|A - B| < 2^{31}$, donc pas de débordement et le signe de la différence est exact. $A < B$ ssi `diff31` = 1. C'est pourquoi A_{31} et B_{31} entrent dans le comparateur.

Slide 37 – Deux motifs seulement.

- **Combinatoire** : `always @(*)` avec des affectations bloquantes `=`. Chaque sortie doit être affectée dans toutes les branches, sinon un latch est inféré. L'astuce : des valeurs par défaut en tête du bloc.
- **Séquentiel** : `always @(posedge clk)` avec des affectations non bloquantes `<=`. Uniquement des registres (avec reset et enable) et des compteurs.

L'exemple de la slide a un reset actif haut et synchrone, alors que le CPU utilise `rst_n` (actif bas). Vérifier dans les guidelines Moodle si le reset attendu est synchrone ou asynchrone.

Slide 38 – Trois `always` par FSM. Logique d'état suivant (combinatoire), registre d'état (séquentiel, `S <= Snext`), logique de sortie (combinatoire). En Mealy, le bloc de sortie lit aussi les entrées. Exemple de logique de sortie Moore pour le contrôleur :

```
always @(*) begin
    ir_en = 0; pc_en = 0; rf_we = 0; sel_addr = 0; sel_mem = 0; // défauts :
    pas de latch
    case (state)
        FETCH2: begin ir_en = 1; pc_en = 1; end
        LOAD1:  sel_addr = 1;
        LOAD2:  begin rf_we = 1; sel_mem = 1; end
    endcase
end
```

Slide 39 – Attention à la simulation à délai nul. En simulation, la sortie d'une bascule change exactement au front. On croit alors que la bascule suivante capture la nouvelle valeur ; elle capture en réalité la valeur d'avant le front. Les `<=` modélisent exactement cela : tous les membres de droite sont évalués avant toute mise à jour. C'est ce qui garantit que, pour `jal`, le RF reçoit `PC + 4` et non la nouvelle cible. Un `=` dans un bloc séquentiel rend le résultat dépendant de l'ordre des lignes. En cas de debug difficile, redessiner le chronogramme à la main avec de petits délais.

7. Résumé et points à maîtriser

La chaîne de raisonnement du cours :

1. Une mémoire synchrone et unifiée impose plusieurs cycles par instruction.
2. On construit le datapath en ajoutant, pour chaque type d'instruction, juste le matériel nécessaire.
3. Chaque ajout crée un signal de contrôle.
4. La FSM est la table « état → valeurs des signaux ».
5. On n'écrit le Verilog qu'une fois tout décomposé.

Points à vérifier avant l'examen et le labo :

- ☐ Le moment où le PC est incrémenté, et ses conséquences sur la base des branches et sur la valeur écrite par `jal / jalr`.
- ☐ Le rôle exact de `sel_pc_base` dans l'énoncé du labo.
- ☐ Le comparateur de l'ALU, signé contre non signé.
- ☐ Le type de reset attendu (synchrone ou asynchrone) dans les guidelines Moodle.

Référence du cours : Patterson & Hennessy, *Computer Organization and Design*, édition RISC-V, chapitre 4, sections 4.1 à 4.5.